

# Scale Space Blob Detectors

Audio Processing, Video Processing and Computer Vision

Lab exercise 2

## Introduction

In this lab, you will implement a scale-space blob detector using the Laplacian of Gaussian (LoG). You will first identify spatial maxima, then select blob scales, and finally build an octave-based detector for images containing blobs of different sizes.

### Exercise 1 Non maximum suppression

Implement non-maximum suppression (NMS) on a synthetic image containing three Gaussian blobs.

Follow these steps:

1. Create a  $100 \times 100$  coordinate grid spanning  $-10$  to  $10$  along both axes. Use `np.linspace` and `np.meshgrid`.

2. Write a function that returns a 2D Gaussian with unit peak height:

```
def gaussian(x, y, x0, y0, sigma):
```

Here,  $(x_0, y_0)$  is the center and  $\sigma$  is the standard deviation in coordinate units.

3. Add three Gaussian blobs with the following centres, standard deviations and peak heights:

Center  $(-4, -4)$ ,  $\sigma = 1.2$ , height = 5.

Center  $(4, -4)$ ,  $\sigma = 1.6$ , height = 10.

Center  $(0, 4)$ ,  $\sigma = 2.2$ , height = 8.

4. Apply non-maximum suppression:

- Use `scipy.ndimage.rank_filter` with `rank=-1` and `size=3` to obtain the maximum in each  $3 \times 3$  neighbourhood.

```
max_filter_output = rank_filter(image, rank=-1, size=3)
```

- Compare the image with `max_filter_output`. Retain pixels equal to the neighbourhood maximum and set the others to zero.

Display the original image, the maximum-filter output and the image after NMS.

### Exercise 2 Blob detection at a single scale

Apply a LoG filter to Sunflowers.jpg. Convert the image to grayscale floating point values in  $[0, 1]$ . Choose sigma (the standard deviation in pixels, not the variance) to match the dark centers of the largest sunflowers.

Filter the image using the LoG filter  
Apply thresholding to the filtered image  
Perform 2D non-maximum suppression  
Display the results

Implementation hints:

- Use `scipy.ndimage.gaussian_laplace` to implement LoG filter.
- Visually inspect selected detections by reviewing the image at each processing stage: filtering, thresholding, and non-maximum suppression.
- Use the supplied `plot_circles(image, cx, cy, rad)` function.

### Exercise 3 Scale selection at full resolution

Continue with Sunflowers.jpg and keep the image resolution fixed. Evaluate the detector at geometrically spaced scales:

$$\sigma(s) = \sigma_0 K^{s/S}, \text{ with } s = 0, 1, \dots, S - 1$$

where  $\sigma_0$  is the initial scale,  $K$  controls the total increase in scale, and  $S$  is the number of scale samples.

Build a response volume before applying NMS across position and scale:

# Build the scale-space

Define  $\sigma_0$ ,  $N_s$  (number of scales), and  $K$

for s in scales

Filter image with the corresponding LoG filter

Threshold and 2D non-maxima suppression

3D non-maxima suppression within each octave

# Displaying the results

Display the results

### Exercise 4 Blob detection with octaves

Continue with Sunflowers.jpg. Group scales into octaves: each octave covers a factor of two in scale, and the image dimensions are halved between octaves. This follows SIFT's octave organisation, but you will retain the LoG detector used above.

The scales, expressed in original-image pixels, are

$$\sigma(o, s) = \sigma_0 2^o 2^{s/S}, \text{ with } s = 0, 1, \dots, S - 1$$

where  $o = 0, 1, \dots, N_o - 1$  is the octave index and  $S$  is the number of scale samples per octave.

### LoG scale space with octaves

Filter at several scales, stack the responses, then select maxima in position and scale.

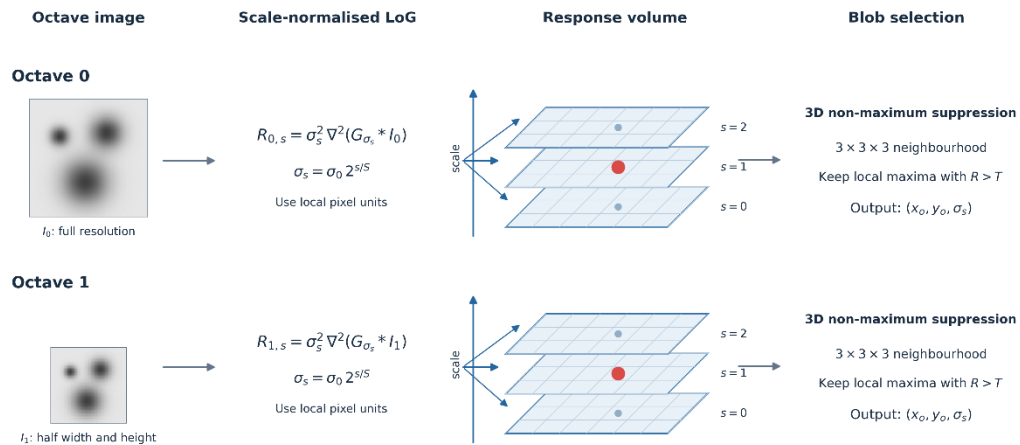


Figure 1. LoG scale space with octaves.

Use the following structure:

# Build the scale-space

Define  $\sigma_0$ ,  $N_o$  (number of octaves), and  $S$

for  $o$  in octaves

Resize the image

for  $sc$  in scales

Filter image with the corresponding LoG filter

Threshold and 2D non-maxima suppression

3D non-maxima suppression within each octave

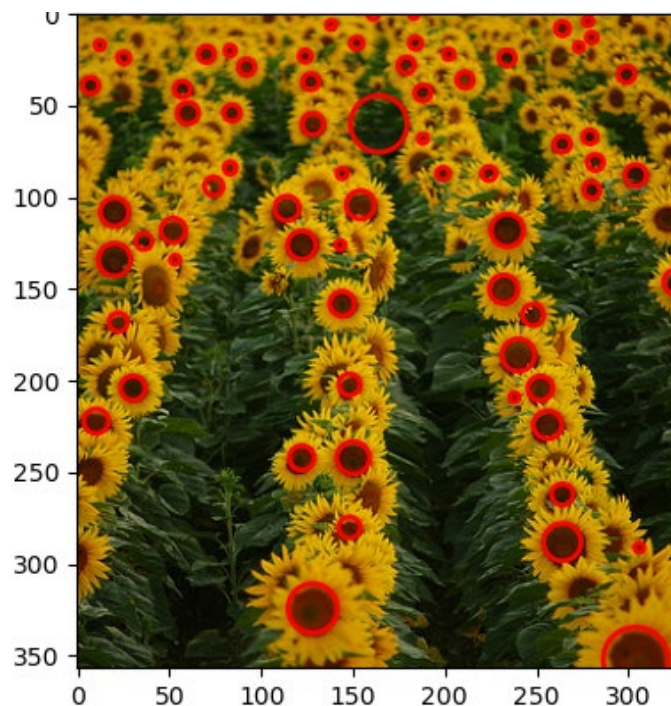
# Displaying the results

Display the results

Implementation hints:

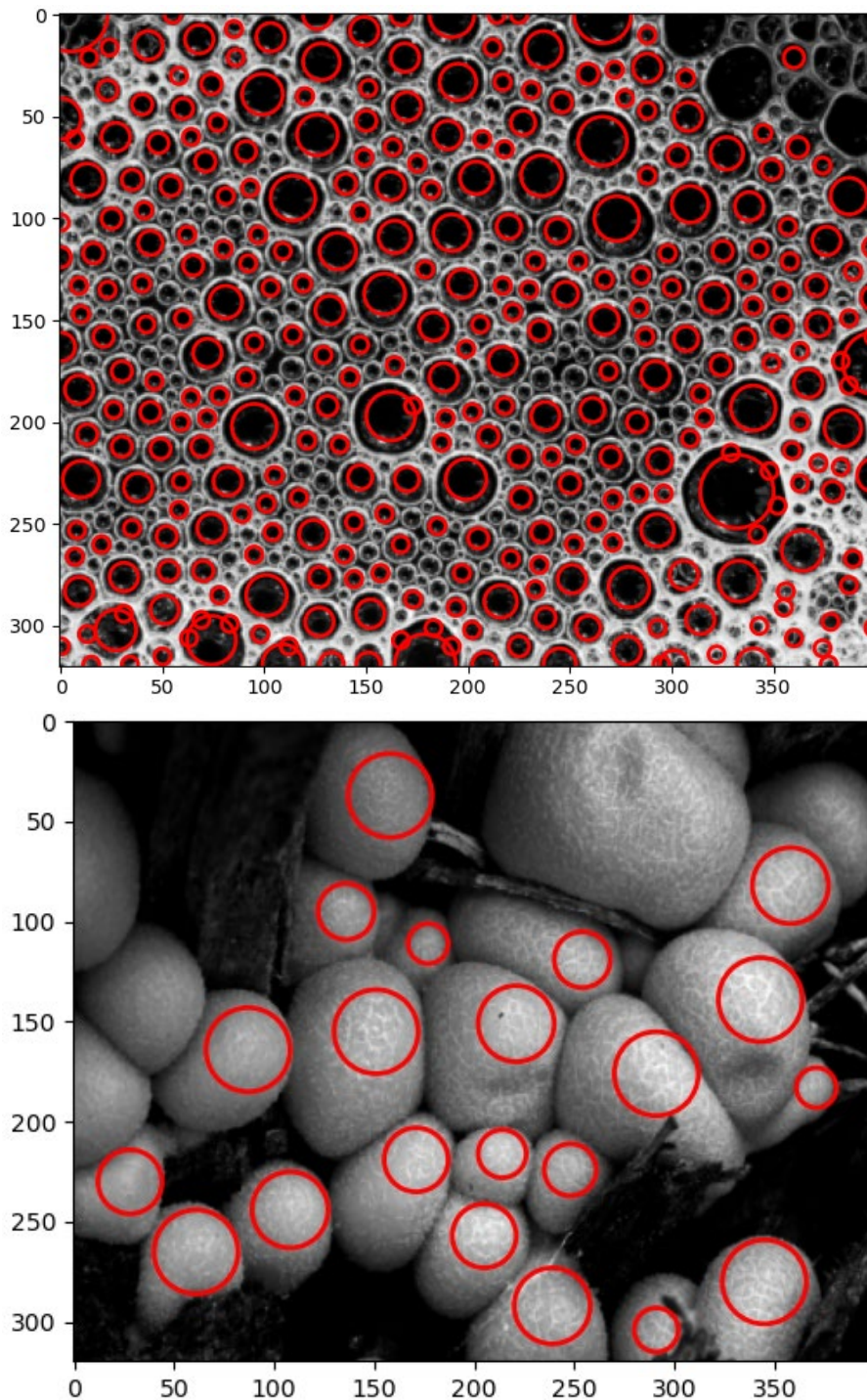
- Estimate the smallest and largest target blobs. Choose  $\sigma_0$ ,  $N_o$  and  $S$  to cover their sizes with roughly 10–15 scale samples in total.
- Use `skimage.transform.resize` when reducing the image size.
- Use the local sigma for filtering and scale normalisation.
- Apply `scipy.ndimage.rank_filter` to the 3D non-maxima suppression within each octave.

The example below illustrates the expected output. Aim to detect centers across the size range; an exact match is not required because results depend on the parameters.



#### Exercise 5 Adapting the detector to other images

Apply your detector to `fruits.jpg` and `water_texture.jpg` (photos from [PxHere](#)). How would you choose the parameters of the algorithm to get these results? The parameterization should be different for each image. Use the examples below as qualitative references.



### Evaluation

The lab will be assessed on October 15th through a 20-minute quiz on the methods implemented in these exercises.